

Data Contract & Decoupling Strategy

Overview

This document defines the standard conventions, naming patterns, and strategies for maintaining consistency across the greenfield_mp data ecosystem. These conventions ensure data quality, interoperability, and maintainability across all layers of the system.

1. Column Naming Conventions

General Rules

- Use **snake_case** for all column names
- Use **descriptive names** that clearly indicate the data's meaning
- Use **singular nouns** for entity names (e.g., `employee_id`, not `employees_id`)
- Avoid abbreviations unless industry-standard (e.g., `pct` for percentage)
- Use **consistent prefixes** for similar concepts

Primary Key Columns

Pattern	Example	Description
<code>{entity}_id</code>	<code>employee_id</code> , <code>project_id</code>	Business key from OLTP
<code>{entity}_key</code>	<code>employee_key</code> , <code>department_key</code>	Surrogate key in dimensions
<code>{entity}_key</code>	<code>review_key</code>	Surrogate key in fact tables

Foreign Key Columns

Pattern	Example	Description
<code>{referenced_entity}_id</code>	<code>department_id</code> , <code>job_role_id</code>	FK to lookup table
<code>{referenced_entity}_key</code>	<code>employee_key</code> , <code>date_key</code>	FK to dimension table

Attribute Columns

Pattern	Example	Description
<code>{entity}_name</code>	<code>department_name</code> , <code>project_name</code>	Descriptive name
<code>{entity}_status</code>	<code>project_status</code> , <code>attrition_flag</code>	Current state
<code>{entity}_level</code>	<code>job_level</code> , <code>education_level</code>	Categorical level
<code>{entity}_pct</code>	<code>allocation_pct</code> , <code>percent_salary_hike</code>	Percentage values

Pattern	Example	Description
{entity}_score	review_score	Computed metric
{entity}_date	hire_date , review_date	Date fields
{entity}_flag	attrition_flag , is_current	Boolean flags (0/1)

Date/Time Columns

Pattern	Example	Description
{entity}_start	effective_start , start_date	Start of validity period
{entity}_end	effective_end , end_date	End of validity period
created_at	created_at	Record creation timestamp
updated_at	updated_at	Last update timestamp

SCD Type 2 Specific

Pattern	Example	Description
is_current	is_current	Flag for current version (0/1)
change_reason	change_reason	Reason for the change

2. Data Types

Standard Data Types by Category

Identifiers

Type	Use For	Example
INT	Primary keys, foreign keys	employee_id , department_id
BIGINT	Large fact table keys	review_key
INT AUTO_INCREMENT	Surrogate keys	employee_key

Text

Type	Use For	Example
VARCHAR(n)	Names, descriptions, status	department_name VARCHAR(60)
TEXT	Large text content	(not used in this project)

Numbers

Type	Use For	Example
TINYINT	Small integers, flags	job_level , attrition_flag
SMALLINT	Medium integers	age , total_working_years
INT	General integers	employee_id , monthly_income
DECIMAL(p,s)	Precise decimals	budget DECIMAL(12,2)

Dates

Type	Use For	Example
DATE	Date values	hire_date , review_date
TIMESTAMP	Creation timestamps	created_at

Booleans

Type	Use For	Example
TINYINT(1)	Boolean flags	is_current , attrition_flag

Type Mapping Table

Concept	MySQL Type	Python Type	Notes
Integer ID	INT	int	Auto-increment primary key
Surrogate Key	INT	int	Dimension table key
Large Key	BIGINT	int	Fact table primary key
Name	VARCHAR(60-100)	str	Descriptive text
Status	VARCHAR(20)	str	Enum-like values
Priority	VARCHAR(10)	str	Low/Medium/High
Rating	TINYINT	int	1-4 scale
Percentage	TINYINT	int	0-100 or 0-50
Score	DECIMAL(5,2)	float	Computed metric
Date	DATE	date	YYYY-MM-DD
Flag	TINYINT(1)	bool	0 = False, 1 = True

3. Primary Key Strategy

OLTP Tables

- Use **auto-increment integer** primary keys
- Business keys stored as separate columns with UNIQUE constraints
- Example: `employees.employee_id` (PK) + `employees.email` (UK)

Dimension Tables

- Use **surrogate keys** (auto-increment integers)
- Business keys preserved for joins and lookups
- Example: `dim_employee.employee_key` (PK) + `dim_employee.employee_id` (business key)

Fact Tables

- Use **auto-increment BIGINT** primary keys
- Degenerate dimensions stored as regular columns
- Example: `fact_performance_reviews.review_key` (PK) + `review_id` (degenerate)

Composite Keys

- Use for junction tables (many-to-many)
- Example: `project_assignments` has `assignment_id` as PK, but unique constraint on (`employee_id`, `project_id`)

4. Foreign Key Strategy

Referential Integrity

- All foreign keys have explicit `CONSTRAINT` definitions
- Use meaningful constraint names: `fk_{table}_{column}`
- Example: `CONSTRAINT fk_emp_department FOREIGN KEY (department_id) REFERENCES departments (department_id)`

Nullable Foreign Keys

- Allow NULL for optional relationships
- Example: `performance_reviews.project_id` can be NULL
- Handle NULLs in ETL with "unknown member" pattern

Unknown Member Pattern

- Create a default row in dimension tables for NULL references

- Example: `dim_project` has row with `project_key = -1`, `project_name = 'No project'`
- This allows foreign key constraints without allowing NULLs in fact tables

Cascade Rules

- **No CASCADE DELETE** - preserve historical data
- **No CASCADE UPDATE** - maintain referential integrity
- Use soft deletes or status flags instead

5. Function Naming Conventions

Python Functions

Pattern	Example	Description
<code>get_{entity}</code>	<code>get_profile()</code> , <code>get_employee()</code>	Retrieve single item
<code>list_{entity}</code>	<code>list_departments()</code> , <code>list_projects()</code>	Retrieve multiple items
<code>add_{entity}</code>	<code>add_employee()</code> , <code>add_project()</code>	Create new item
<code>update_{entity}</code>	<code>update_employee()</code>	Modify existing item
<code>delete_{entity}</code>	<code>delete_employee()</code>	Remove item
<code>change_{entity}</code>	<code>change_department()</code>	Specific action
<code>submit_{entity}</code>	<code>submit_review()</code>	Submit for processing
<code>compute_{metric}</code>	<code>compute_score()</code>	Calculate derived value

SQL Stored Procedures

Pattern	Example	Description
<code>sp_{action}_{target}</code>	<code>sp_load_oltp_from_staging()</code>	ETL procedures
<code>sp_load_dim_{entity}</code>	<code>sp_load_dim_employee_scd2()</code>	Dimension load
<code>sp_load_fact_{entity}</code>	<code>sp_load_fact_performance_reviews()</code>	Fact load
<code>sp_run_full_etl</code>	<code>sp_run_full_etl()</code>	Master ETL procedure

SQL Functions

Pattern	Example	Description
<code>{verb}_{noun}</code>	<code>compute_review_score()</code>	Descriptive action

6. API Naming Conventions

REST API Endpoints

Pattern	Example	Description
GET /api/{resource}	GET /api/employees	List all
GET /api/{resource}/{id}	GET /api/employees/123	Get single
POST /api/{resource}	POST /api/employees	Create
PUT /api/{resource}/{id}	PUT /api/employees/123	Update
DELETE /api/{resource}/{id}	DELETE /api/employees/123	Delete
PATCH /api/{resource}/{id}	PATCH /api/employees/123	Partial update

Query Parameters

Pattern	Example	Description
{field}=value	?department_id=5	Filter by field
{field}={operator}{value}	?created_at>=2025-01-01	Range filter
sort={field}	?sort=name	Sort results
limit={n}	?limit=100	Pagination
offset={n}	?offset=200	Pagination

Response Structure

<pre>{ "data": [...], "meta": { "total": 1000, "limit": 100, "offset": 0 } }</pre>
--

7. ETL Naming Conventions

Staging Tables

Pattern	Example	Description
stg_{source}_{entity}	stg_employee_snapshot , stg_employee_history	Raw data from source

ETL Scripts

Pattern	Example	Description
<code>{verb}_{noun}.py</code>	<code>load_staging.py</code> , <code>run_etl.py</code>	Script purpose
<code>{noun}_manager.py</code>	<code>db_manager.py</code>	Module name

ETL Procedures

Pattern	Example	Description
<code>sp_{verb}_{target}</code>	<code>sp_load_oltp_from_staging()</code>	Load operation
<code>sp_{verb}_{target}_{from}</code>	<code>sp_load_oltp_from_staging()</code>	Source specification

ETL Variables

Pattern	Example	Description
<code>p_{param_name}</code>	<code>p_employee_id</code> , <code>p_start_date</code>	Procedure parameters
<code>v_{var_name}</code>	<code>v_current_version</code>	Local variables
<code>tmp_{table_name}</code>	<code>tmp_emp_versions</code>	Temporary tables

8. Versioning Strategy

Database Schema Versioning

- Use **semantic versioning** for schema changes: `v{major}.{minor}.{patch}`
- Track schema version in a `schema_version` table
- Example: `INSERT INTO schema_version (version, applied_at) VALUES ('1.2.0', NOW())`

API Versioning

- Use **URL versioning**: `/api/v1/employees`
- Default to latest stable version
- Deprecate old versions with 6-month notice

Data Model Versioning

- **SCD Type 2** for slowly changing dimensions
- **Type 1** for overwritten attributes
- **Type 3** for limited history (not used in this project)

Release Versioning

- Follow **Semantic Versioning 2.0.0**: `MAJOR.MINOR.PATCH`
- **MAJOR**: Breaking changes
- **MINOR**: New features, backward compatible
- **PATCH**: Bug fixes, backward compatible

Branch Versioning

Branch Type	Version Pattern	Example
Release	<code>release/v{major}.{minor}</code>	<code>release/v1.0</code>
Hotfix	<code>hotfix/v{major}.{minor}.{patch}</code>	<code>hotfix/v1.0.1</code>
Feature	<code>feature/{description}</code>	<code>feature/employee-search</code>

9. Data Quality Standards

Validation Rules

1. **Not Null**: Required fields must not be NULL
2. **Unique**: Business keys must be unique
3. **Check Constraints**: Value ranges enforced at database level
4. **Foreign Key**: Referential integrity enforced
5. **Default Values**: Sensible defaults for optional fields

Data Cleansing

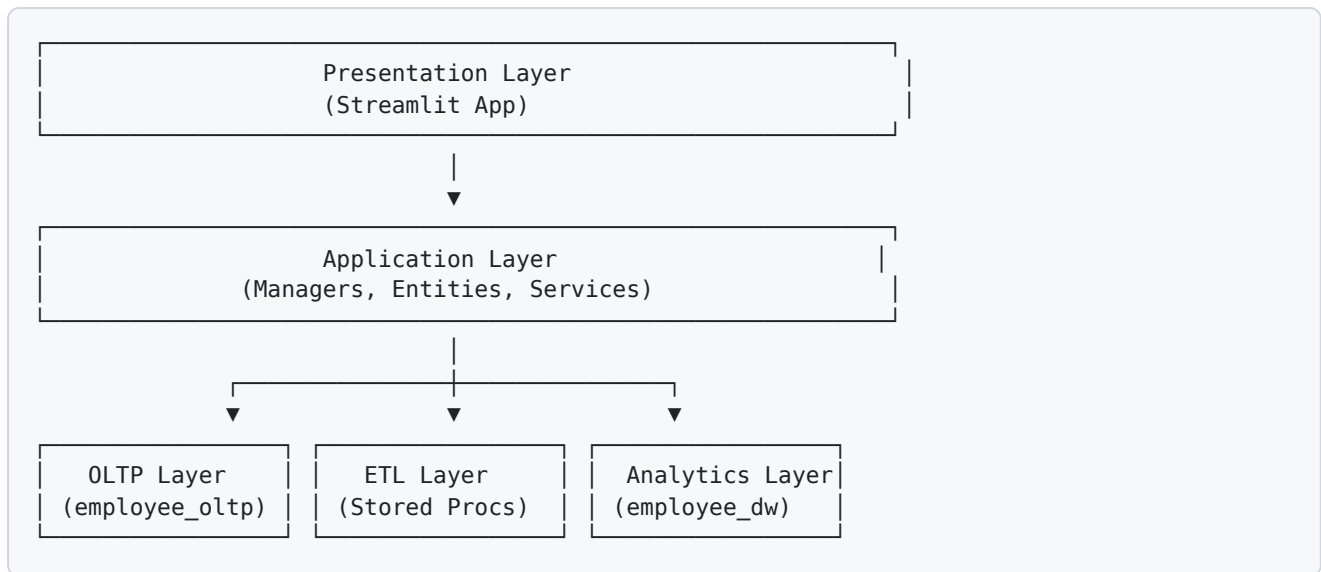
1. **Trim**: Remove leading/trailing whitespace
2. **Lowercase**: Normalize case for emails
3. **Standardize**: Convert to consistent format
4. **Validate**: Check against expected patterns

Error Handling

- Raise `ValueError` for business rule violations
- Raise `DatabaseError` for database errors
- Log all data quality issues
- Provide clear error messages to users

10. Decoupling Strategy

Layer Separation



Data Contracts Between Layers

1. **OLTP → ETL**: Use stored procedures for data movement
2. **ETL → Warehouse**: Surrogate keys for dimension tables
3. **Warehouse → Analytics**: Star schema with denormalized dimensions

Interface Contracts

- **Managers** provide clean APIs to entities
- **Entities** encapsulate validation logic
- **DatabaseConnection** abstracts connection details
- **Stored Procedures** encapsulate ETL logic

Backward Compatibility

- Never remove columns from production tables
- Use additive changes only
- Deprecate columns with comments
- Maintain API compatibility across minor versions

11. Naming Convention Quick Reference

Tables

Type	Pattern	Example
Staging	stg_{source}_{entity}	stg_employee_snapshot
OLTP	{entity}	employees , projects
Dimension	dim_{entity}	dim_employee , dim_date
Fact	fact_{subject}	fact_performance_reviews

Columns

Type	Pattern	Example
PK	{entity}_id or {entity}_key	employee_id , employee_key
FK	{entity}_id or {entity}_key	department_id , date_key
Name	{entity}_name	department_name
Status	{entity}_status	project_status
Date	{entity}_date	review_date
Flag	{entity}_flag	is_current

Files

Type	Pattern	Example
Python	{noun}_manager.py	db_manager.py
SQL	{number}_{purpose}.sql	01_schemas_and_staging.sql
Scripts	{verb}_{noun}.py	load_staging.py
Docs	{topic}-guide.md	setup-guide.md